

SIGma Fall 2026

Welcome (Back)!

Ian Chen



Outline

Admins (in No Particular Order)

Computational Geometry

CG:SHOP

Meetings



Housekeeping



- Join the Discord!



Section 1

Admins (in No Particular Order)



Ian

- Senior in Stats & CS
- CS 473 course assistant
- General interests: graph algorithms, parallel computing, algorithmic engineering



Mihir

•



Franklin

•



Section 2

Computational Geometry



Applications

- Robotics; TAMP
- Computer graphics
- Geographic information systems (GIS)
- Computer aided {design, manufacturing} (CAD, CAM)



Convex hulls.



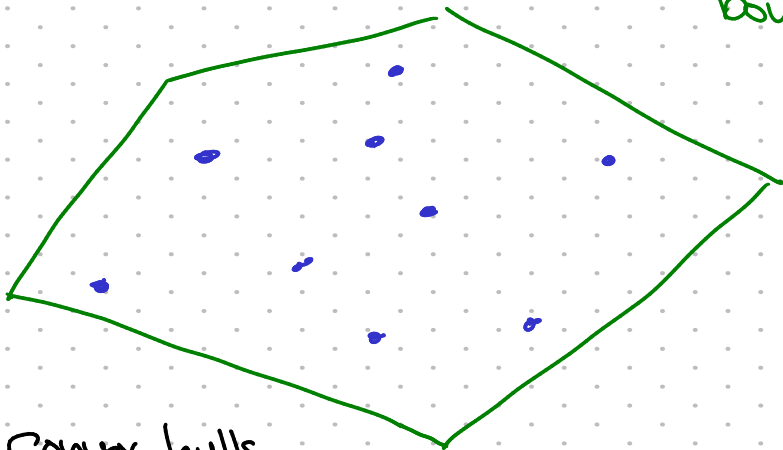
points in 2D space



Convex hulls.



points in 2D space
bounding polygon

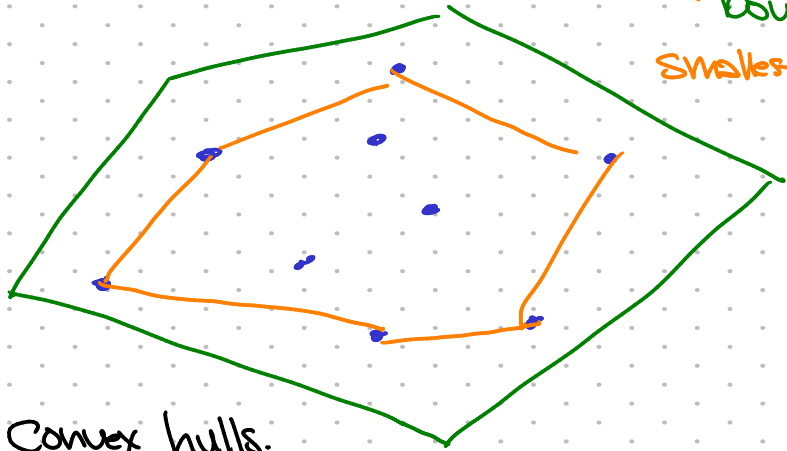


Convex hulls.



points in 2D space

✓ bounding polygon
Smallest "convex"



Convex hulls.



Convex hulls

- Input: n points in 2D space
- Output smallest convex bounding polygon

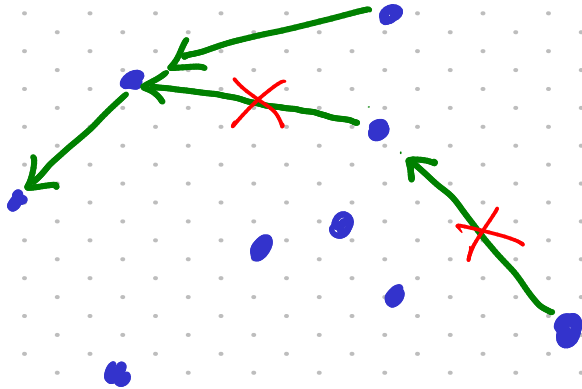


Convex hulls

- Input: n points in 2D space
 - ▶ an unordered list of (x, y) pairs
- Output smallest convex bounding polygon
 - ▶ an ordered list of (x, y) pairs
 - ▶ subset of the input; in CCW order

```
Point = namedtuple("Point", ["x", "y"])  
def upper_hull(points: set[Point]) -> list[Point]:  
    ...
```



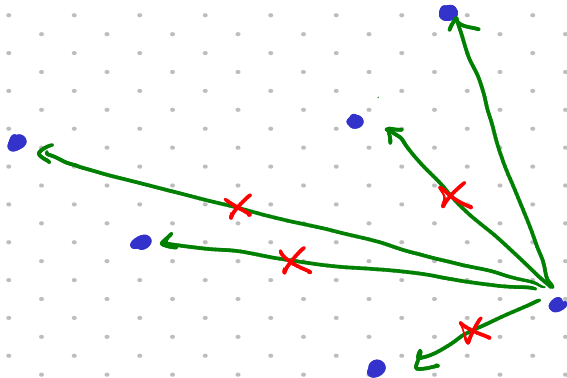


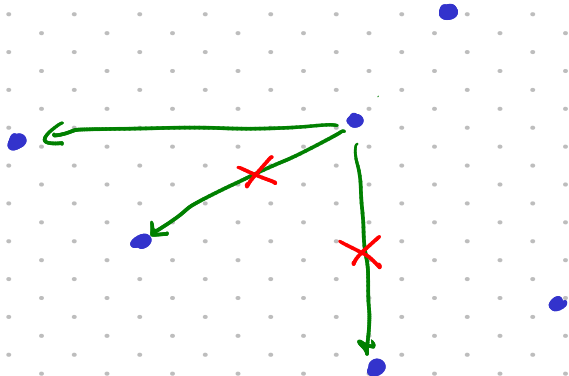
Σ

Brute Force

```
def upper_hull_1(points: set[Point]) -> list[Point]:  
    lines = []  
    for p1, p2 in combinations(points, 2):  
        p1, p2 = max(p1, p2), min(p1, p2)  
        if all(point_below_line((p1, p2), q)  
                for q in points):  
            lines.append((p1, p2))  
    return sorted(set(flatten(lines)), reverse=True)
```





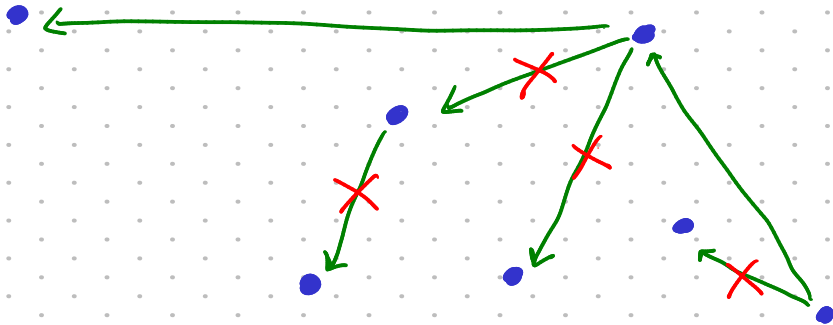


Σ

A little smarter...

```
def upper_hull_2(points: set[Point]) -> list[Point]:  
    lines = []  
    for p1 in points - {min(points)}:  
        p2 = max((q for q in points if q < p1),  
                  key=lambda q: sin_angle(p1, q))  
        if all(point_below_line((p1, p2), q)  
                for q in points):  
            lines.append((p1, p2))  
    return sorted(set(flatten(lines)), reverse=True)
```





Sweeping

```
def upper_hull_3(points: set[Point]) -> list[Point]:
    hull = []
    for point in sorted(points, reverse=True):
        while len(hull) >= 2 and
            not point_below_line(
                (hull[-2], hull[-1]), point):
            hull.pop()
        hull.append(point)
    return hull
```



Section 3

CG:SHOP



- For mihir to prepare slides for and present



Section 4

Meetings



Standard meetings

- $\approx$ 30 minutes for a *lecture*, plus questions
 - ▶ Present a problem, or approaches, in computational geometry
 - ▶ Spreadsheet for presentation signups: TODO
 - ▶ Admins are very willing to help with any presenters.
 - ▶ Resources: jeffe.cs.illinois.edu/teaching/compgeom/resources.html
- $\approx$ 10 minutes for project updates and discussion



Project centered meetings

- TODO: what happens during this?
- algorithms focused team, implementation focused team
- but lots of overlap between them



Project AI Policy

- TODO:

